

① Star Patterns :-

(a) Square :-

Logic :

- Use two nested loops.
- The outer loop controls the rows (running n times).
- The inner loop controls the columns (running n times).
- Print $*$ in every iteration of inner loop and move to a new line after the inner loop finishes.

Code :

```
int n = 5;  
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        System.out.println(" * ");  
    }  
    System.out.println();  
}
```

(b) Hollow Square :-

Logic :

- Use two nested loops running n times each.
- Check the current row and column indices.
- If it is the first row, last row, first

- column, or last column, print *
- Otherwise, print a blank space " " to create the hollow center.

Code :

```
int n = 5;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        if (i == 0 || i == n - 1 || j == 0 || j == n - 1) {
            System.out.println("*");
        }
        else {
            System.out.println(" ");
        }
    }
    System.out.println();
}
```

(c) Triangle :-

Logic :-

- Use an outer loop for n rows.
- Use an inner loop that runs from 0 to up to the current row index i.
- Print * in the inner loop to gradually increase the number of stars per row.

Code :

```

int n = 5;
for (int i = 0; i < n; i++) {
    for (int j = 0; j <= i; j++) {
        System.out.print("*");
    }
    System.out.println();
}

```

(d) Inverted Triangle:-Logic :

- Use an outer loop to track the rows, running n times (from 0 to $n-1$).
- Use a first inner loop to print the leading spaces. The number of spaces starts at 0 and increases by 1 on each new row (i spaces).
- Use a second inner loop to print the stars. To keep the triangle centered, the stars must decrease by 2 each time.
- Move to the next line after both inner loops finish.

Code :

```

int n = 5;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        System.out.print(" ");
    }
}

```



```

    }
    for (int j = 0; j < n - i; j++) {
        System.out.println(" * ");
    }
}

```

```

    System.out.println();
}

```

(e) Pyramid :-

Logic :-

- Use an outer loop for n rows.
- Use a first inner loop to print spaces.
- The number of spaces decrease as the row number increases ($n - i - 1$).
- Use a second inner loop to print stars.
- The number of stars follow odd numbers.

Code :

```

int n = 5;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n - i - 1; j++) {
        System.out.println(" ");
    }
    for (int j = 0; j < 2 * i + 1; j++) {
        System.out.print(" * ");
    }
    System.out.println();
}

```


(f) Diamond :-

Logic :

- Combine a standard pyramid logic for the upper half and an inverted pyramid logic for the lower half.
- The upper half loop runs n times.
- The lower half loop runs $n-1$ times, with spaces increasing and stars decreasing.

Code :

```

int n = 5;

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n - i - 1; j++) {
        System.out.print(" ");
    }
    for (int j = 0; j < 2 * i + 1; j++) {
        System.out.print("*");
    }
    System.out.print("\n");
}

for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < i; j++) {
        System.out.print(" ");
    }
    for (int j = 0; j < 2 * (n - i - 1) - 1; j++) {
        System.out.print("*");
    }
}

```



```

    System.out.println();
}

```

(g) Half Diamond :-

Logic :

- Create an increasing right triangle for the upper half.
- Create decreasing inverted right triangle for the lower half.
- The upper loop increases stars from 1 to n. The lower loop decreases stars from n-1 to 1.

Code :

```

int n = 5;
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print("*");
    }
    System.out.println();
}
for (int i = n-1; i >= 1; i--) {
    for (int j = 1; j <= i; j++) {
        System.out.print("*");
    }
    System.out.println();
}

```


(h) Half Diamond Inverted :-Logic :

- Use two main segments : Upper and lower.
- In each segment, use an inner loop to print the required leading spaces, followed by a loop to print the stars.

Code :-

```

int n = 5;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        System.out.print(" ");
    }
    for (int j = 0; j < n - i; j++) {
        System.out.print("*");
    }
    System.out.println();
}

```


② Number Patterns :-

(a) Square :-

Logic :

- Use two nested loops, both running n -times.
- Instead of a star, strictly print the number 1 during every iteration of inner loop.

Code :

```
int n = 5;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        System.out.print("1");
    }
    System.out.println();
}
```

(b) Incrementing :-

Logic :

- Use an outer loop for n rows.
- Use an inner loop that runs from 1 to n .
- Print the value of inner loop variable to create an incrementing sequence on each row.

Code :

```
int n = 5;
for (int i = 0; i < n; i++) {
    for (int j = 1; j <= n; j++) {
        System.out.print(j);
    }
    System.out.println();
}
```

(c) Right Triangle :-

Logic :

Use an outer loop running from 1 to n to represent the row number.
Use an inner loop running from 1 to the current row number.
Print the inner loop's counter variable.

Code :

```
int n = 5;
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print(j);
    }
    System.out.println();
}
```


Diamond

(d) Inverted Triangle :-Logic :

- Use an outer loop to track the current row, running n times.
- Use a first inner loop to print spaces.
- The number of spaces starts at 0 and increases by 1 for each row (i).
- Use a second inner loop to print the number followed by a space ($i + " "$).
- The amount of times the number is printed decreases by 1 each row.
- Move to the next line after both inner loops complete.

Code :

```
for (int i = 3; i <= 7; i++) {
    for (int j = 0; j < i - 3; j++) {
        System.out.print(" ");
    }
```

```
    for (int j = 0; j < 8 - i; j++) {
        System.out.print(i + " ");
    }
```

```
    System.out.println();
}
```


(e) Sandwich Pattern :-

Logic :

- Split the logic into two parts : an inverted upper triangle (from 3 down to 5) and a regular triangle underneath it (from 4 down to 3).
- The outer loop runs i from 3 to 5, in the case of upper half.
- The outer loop runs backwards, with i from 4 down to 3, in the case of lower half.

Code :

```

for (int i = 3; i <= 5; i++) {
    for (int j = 0; j < i - 3; j++) {
        System.out.print(" ");
    }
    for (int j = 0; j < 2 * (5 - i) + 1; j++) {
        System.out.print("X");
    }
    System.out.println();
}

```

```

for (int i = 4; i >= 3; i--) {
    for (int j = 0; j < i - 3; j++) {
        System.out.print(" ");
    }
}

```



```

for (int i = 0; i < 2 * (5 - i) + 1; i++) {
    System.out.print (i + " ");
}
System.out.println();
}

```

(f) Incrementing Diamond Pattern :-

Logic :

- Notice that the numbers are aligned to the left, but they have spaces between them. Split the pattern into an expanding top half and a shrinking bottom half.
- The outer loop runs i from 3 to 1 i.e. upper half.
- The outer loop runs backwards, with i from 6 down to 3 i.e. lower half.

Code :

```

for (int i = 3; i <= 7; i++) {
    for (int j = 0; j < i - 2; j++) {
        System.out.print (i + " ");
    }
    System.out.println();
}

```

```

for (int i = 6; i >= 3; i--) {
    for (int j = 0; j < i - 2; j++) {

```



```

{
    System.out.print(i + "\n");
}
System.out.println();
}

```

1. Accessing the array elements

Ex:

Let's say we have an array of 10 elements. We want to access the first element. We can do this by using the index 0. The first element is at index 0, the second at index 1, and so on.

Let's say we have an array of 10 elements. We want to access the first element. We can do this by using the index 0. The first element is at index 0, the second at index 1, and so on.

Let's say we have an array of 10 elements. We want to access the first element. We can do this by using the index 0. The first element is at index 0, the second at index 1, and so on.

Ex:

```

{
    for (int i = 0; i < arr.length; i++)
    {
        System.out.print(arr[i] + " ");
    }
    System.out.println();
}

```

```

{
    for (int i = 0; i < arr.length; i++)
    {
        System.out.print(arr[i] + " ");
    }
    System.out.println();
}

```


OUTPUT

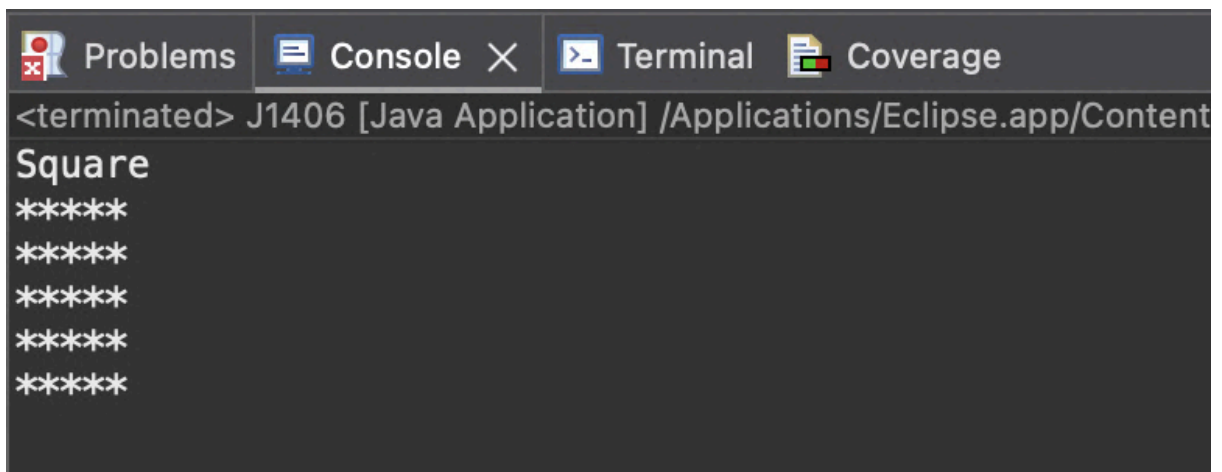
1. STAR PATTERNS:

- SQUARE:

CODE:

```
public class J1406 {  
    public static void main(String[] args) {  
        System.out.println("Square");  
        int n = 5;  
        for (int i = 0; i < n; i++) {  
            for (int j = 0; j < n; j++) {  
                System.out.print("*");  
            }  
            System.out.println();  
        }  
    }  
}
```

OUTPUT:



The screenshot shows the Eclipse IDE interface with the 'Console' tab selected. The console output displays the word 'Square' followed by five lines of five asterisks each, forming a square pattern. The top of the IDE shows tabs for 'Problems', 'Console', 'Terminal', and 'Coverage'. The console title bar indicates the application is 'J1406 [Java Application]' running in the path '/Applications/Eclipse.app/Content'.

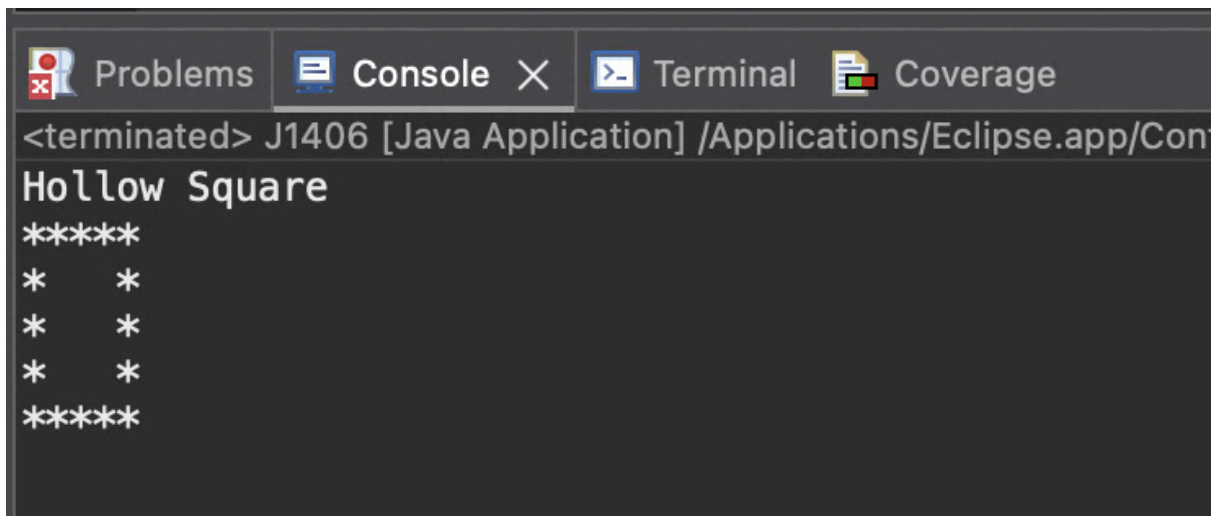
```
<terminated> J1406 [Java Application] /Applications/Eclipse.app/Content  
Square  
*****  
*****  
*****  
*****  
*****
```


- **HOLLOW SQUARE:**

CODE:

```
public class J1406 {  
    public static void main(String[] args) {  
        System.out.println("Hollow Square");  
        int n = 5;  
        for (int i = 0; i < n; i++) {  
            for (int j = 0; j < n; j++) {  
                if (i == 0 || i == n - 1 || j == 0 || j == n - 1) {  
                    System.out.print("*");  
                } else {  
                    System.out.print(" ");  
                }  
            }  
            System.out.println();  
        }  
    }  
}
```

OUTPUT:



The screenshot shows the Eclipse IDE's console window. The title bar includes tabs for 'Problems', 'Console', 'Terminal', and 'Coverage'. The 'Console' tab is active, displaying the output of the Java application. The output starts with the text '<terminated> J1406 [Java Application] /Applications/Eclipse.app/Con'. Below this, the program's output is displayed: 'Hollow Square' followed by a 5x5 hollow square pattern of asterisks. The pattern consists of a top row of 5 asterisks, four middle rows with asterisks at the first and last positions, and a bottom row of 5 asterisks.

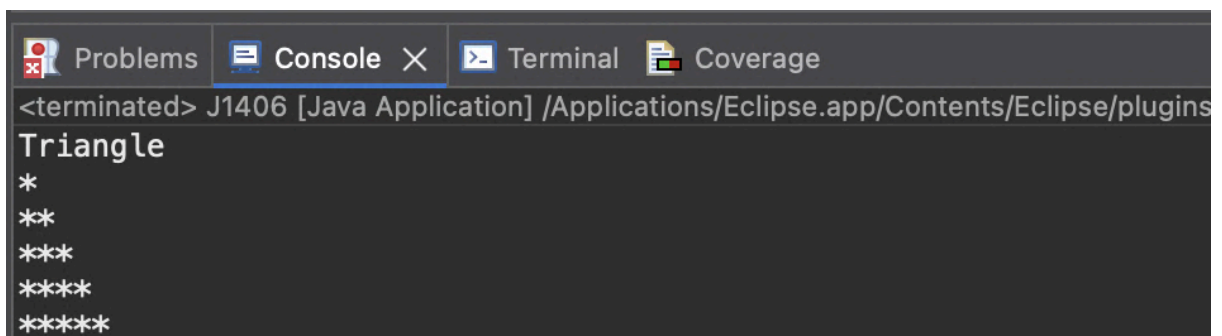
```
<terminated> J1406 [Java Application] /Applications/Eclipse.app/Con  
Hollow Square  
*****  
*      *  
*      *  
*      *  
*****
```


- TRIANGLE:

CODE:

```
public class J1406 {  
    public static void main(String[] args) {  
        System.out.println("Triangle");  
        int n = 5;  
        for (int i = 0; i < n; i++) {  
            for (int j = 0; j <= i; j++) {  
                System.out.print("*");  
            }  
            System.out.println();  
        }  
    }  
}
```

OUTPUT:



The screenshot shows the Eclipse IDE's Console window. The title bar includes tabs for Problems, Console, Terminal, and Coverage. The Console tab is active, displaying the output of the Java application J1406. The output consists of the word "Triangle" followed by five lines of asterisks forming a right-angled triangle: one asterisk on the first line, two on the second, three on the third, four on the fourth, and five on the fifth.

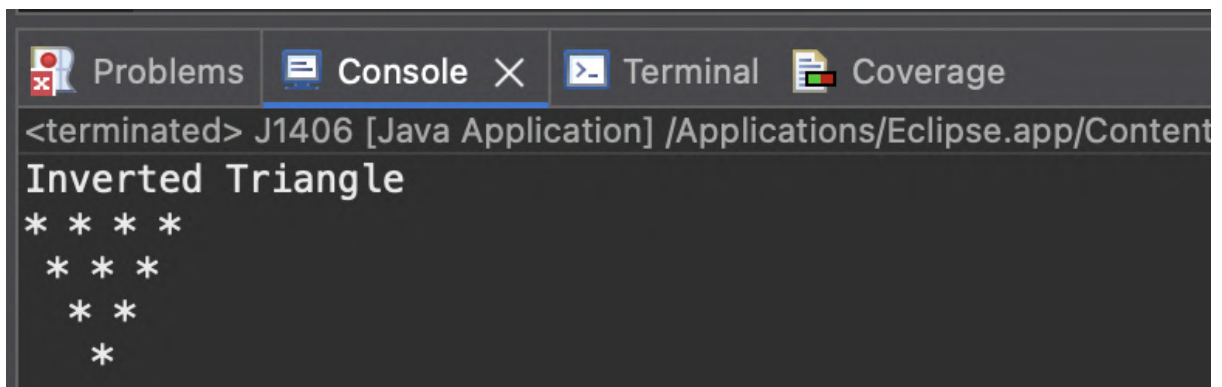
```
<terminated> J1406 [Java Application] /Applications/Eclipse.app/Contents/Eclipse/plugins  
Triangle  
*  
**  
***  
****  
*****
```


- INVERTED TRIANGLE:

CODE:

```
public class J1406 {  
    public static void main(String[] args) {  
        System.out.println("Inverted Triangle");  
        int n = 5;  
  
        for (int i = 0; i < n; i++) {  
            for (int j = 0; j < i; j++) {  
                System.out.print(" ");  
            }  
            for (int j = 0; j < (n - i) - 1; j++) {  
                System.out.print("* ");  
            }  
            System.out.println();  
        }  
    }  
}
```

OUTPUT:



The screenshot shows the Eclipse IDE's Console window. The title bar includes tabs for Problems, Console, Terminal, and Coverage. The Console tab is active, displaying the output of the Java application J1406. The output consists of the text "Inverted Triangle" followed by a pattern of asterisks forming an inverted triangle with 5 rows: 4 spaces and 4 stars, 3 spaces and 3 stars, 2 spaces and 2 stars, 1 space and 1 star, and finally 0 spaces and 1 star.

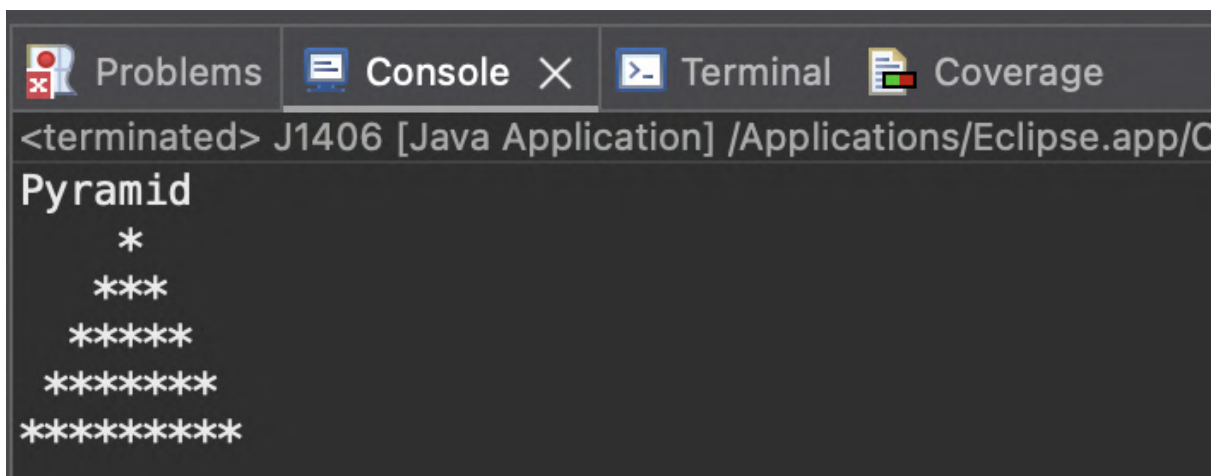
```
<terminated> J1406 [Java Application] /Applications/Eclipse.app/Content  
Inverted Triangle  
* * * *  
 * * *  
  * *  
   *  
    *
```


- PYRAMID:

CODE:

```
public class J1406 {  
    public static void main(String[] args) {  
        System.out.println("Pyramid");  
        int n = 5;  
        for (int i = 0; i < n; i++) {  
            for (int j = 0; j < n - i - 1; j++) {  
                System.out.print(" ");  
            }  
            for (int j = 0; j < 2 * i + 1; j++) {  
                System.out.print("*");  
            }  
            System.out.println();  
        }  
    }  
}
```

OUTPUT:



The screenshot shows the Eclipse IDE interface with the Console tab selected. The console output displays the word "Pyramid" followed by a pyramid shape made of asterisks. The pyramid has 5 rows, with the number of asterisks increasing by 2 in each row (1, 3, 5, 7, 9). There are spaces before the asterisks in each row to center them.

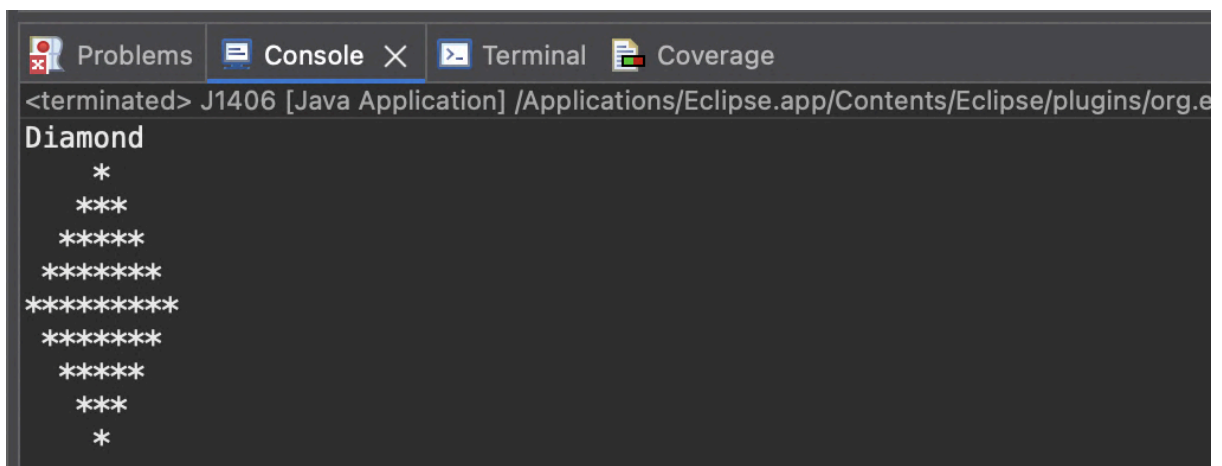
```
<terminated> J1406 [Java Application] /Applications/Eclipse.app/C  
Pyramid  
    *  
   ***  
  *****  
 *****  
*****
```


- DIAMOND:

CODE:

```
public class J1406 {  
    public static void main(String[] args) {  
        System.out.println("Diamond");  
        int n = 5;  
        for (int i = 0; i < n; i++) {  
            for (int j = 0; j < n - i - 1; j++) {  
                System.out.print(" ");  
            }  
            for (int j = 0; j < 2 * i + 1; j++) {  
                System.out.print("*");  
            }  
            System.out.println();  
        }  
        for (int i = 0; i < n - 1; i++) {  
            for (int j = 0; j <= i; j++) {  
                System.out.print(" ");  
            }  
            for (int j = 0; j < 2 * (n - i - 1) - 1; j++) {  
                System.out.print("*");  
            }  
            System.out.println();  
        }  
    }  
}
```

OUTPUT:



The screenshot shows the Eclipse IDE interface with the 'Console' tab selected. The console output displays the word 'Diamond' followed by a diamond-shaped pattern of asterisks. The pattern consists of two parts: an upper triangle and a lower inverted triangle, both with a width of 5 characters. The upper triangle has 5 rows, and the lower inverted triangle has 4 rows. The total output is as follows:

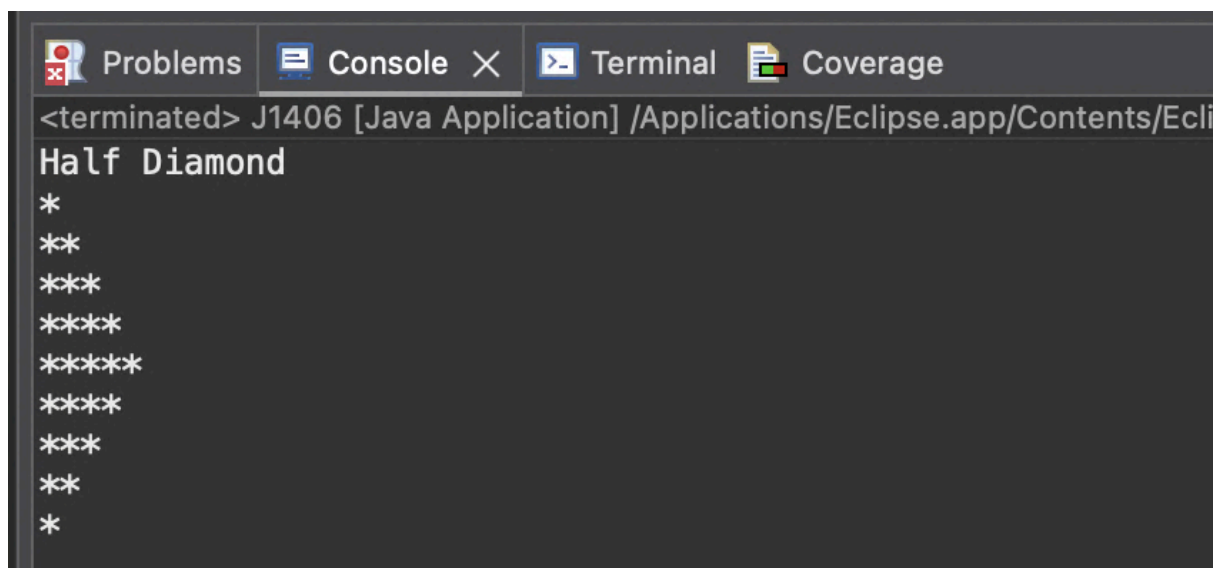
```
<terminated> J1406 [Java Application] /Applications/Eclipse.app/Contents/Eclipse/plugins/org.e  
Diamond  
    *  
   ***  
  *****  
 *****  
*****  
*****  
 *****  
  *****  
   ***  
    *
```


- HALF DIAMOND:

CODE:

```
public class J1406 {  
    public static void main(String[] args) {  
        System.out.println("Half Diamond");  
        int n = 5;  
        for (int i = 1; i <= n; i++) {  
            for (int j = 1; j <= i; j++) {  
                System.out.print("*");  
            }  
            System.out.println();  
        }  
        for (int i = n - 1; i >= 1; i--) {  
            for (int j = 1; j <= i; j++) {  
                System.out.print("*");  
            }  
            System.out.println();  
        }  
    }  
}
```

OUTPUT:



The screenshot shows the Eclipse IDE's console window. The title bar includes tabs for 'Problems', 'Console', 'Terminal', and 'Coverage'. The console output shows the program's execution: it prints 'Half Diamond' followed by a diamond shape made of asterisks. The diamond has 5 rows of increasing width (1 to 5 asterisks) and 5 rows of decreasing width (5 to 1 asterisk). The console text is as follows:

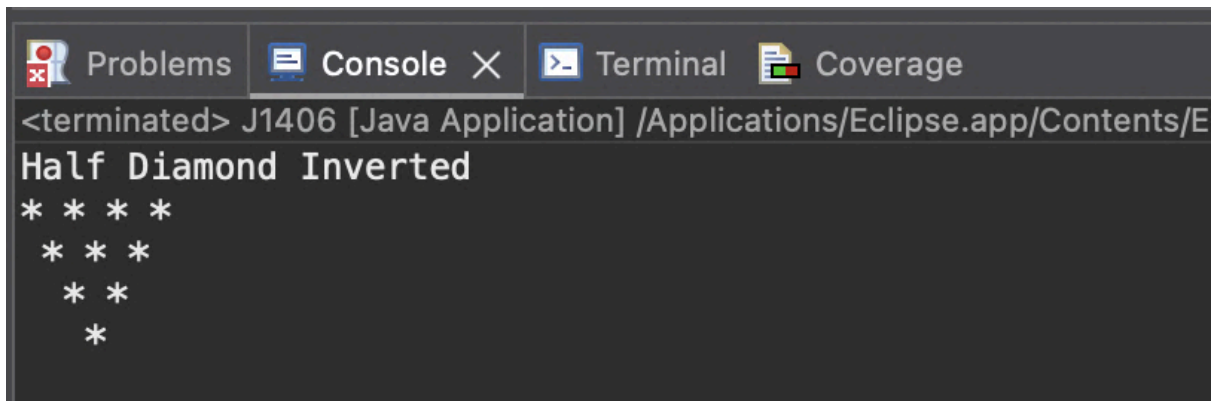
```
<terminated> J1406 [Java Application] /Applications/Eclipse.app/Contents/Ecli  
Half Diamond  
*  
**  
***  
****  
*****  
*****  
****  
***  
**  
*
```


- HALF DIAMOND INVERTED:

CODE:

```
public class J1406 {  
    public static void main(String[] args) {  
        System.out.println("Half Diamond Inverted");  
        int n = 5;  
        for (int i = 0; i < n; i++) {  
            for (int j = 0; j < i; j++) {  
                System.out.print(" ");  
            }  
            for (int j = 0; j < (n - i) - 1; j++) {  
                System.out.print("* ");  
            }  
            System.out.println();  
        }  
    }  
}
```

OUTPUT:



The screenshot shows the Eclipse IDE interface with the 'Console' tab selected. The console output displays the title 'Half Diamond Inverted' followed by a pattern of asterisks. The pattern consists of four lines: the first line has four asterisks, the second has three, the third has two, and the fourth has one. This represents an inverted half-diamond shape.

```
<terminated> J1406 [Java Application] /Applications/Eclipse.app/Contents/E  
Half Diamond Inverted  
* * * *  
 * * *  
  * *  
   *
```

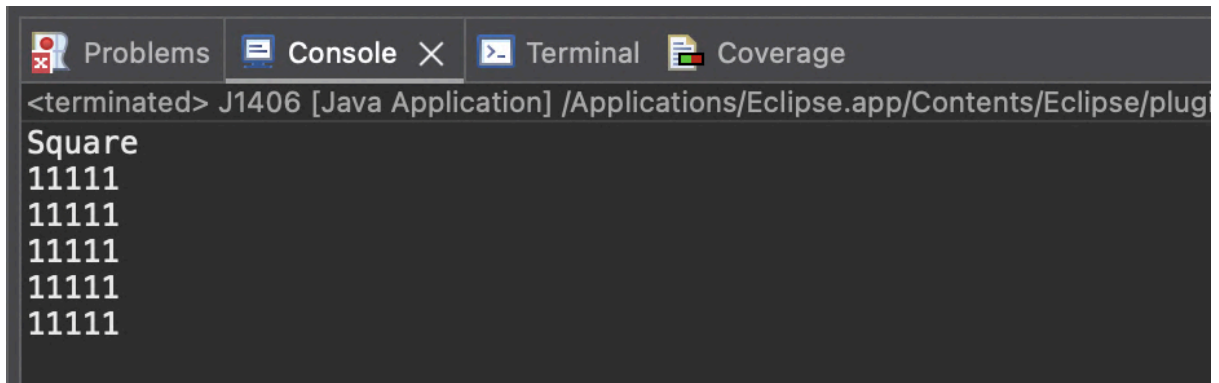

2. NUMBER PATTERNS:

- SQUARE:

CODE:

```
public class J1406 {  
    public static void main(String[] args) {  
        System.out.println("Square");  
        int n = 5;  
        for (int i = 0; i < n; i++) {  
            for (int j = 0; j < n; j++) {  
                System.out.print("1");  
            }  
            System.out.println();  
        }  
    }  
}
```

OUTPUT:



The screenshot shows the Eclipse IDE interface with the 'Console' tab selected. The console output displays the word 'Square' followed by a 5x5 grid of the digit '1'. The IDE title bar indicates the application is 'J1406 [Java Application]' located at '/Applications/Eclipse.app/Contents/Eclipse/plug'.

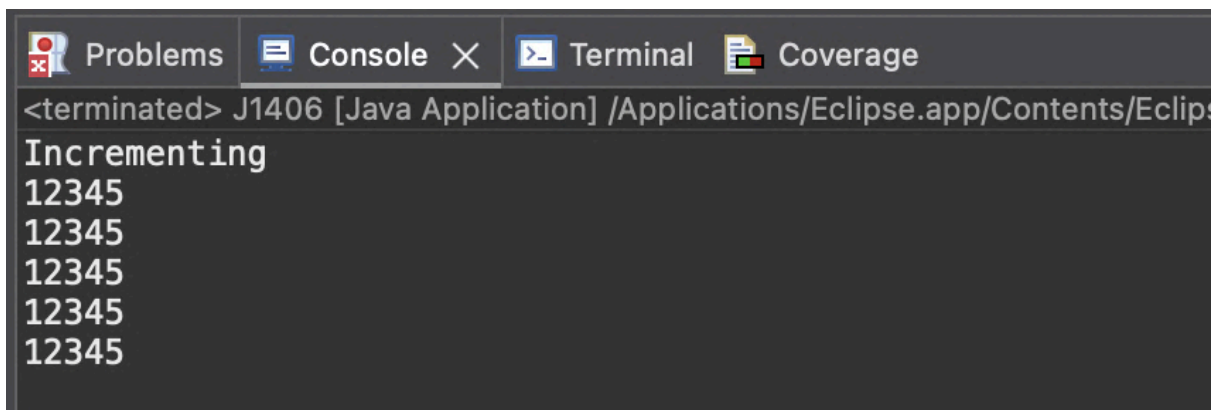
```
<terminated> J1406 [Java Application] /Applications/Eclipse.app/Contents/Eclipse/plug  
Square  
11111  
11111  
11111  
11111  
11111
```


- INCREMENTING:

CODE:

```
public class J1406 {  
    public static void main(String[] args) {  
        System.out.println("Incrementing");  
        int n = 5;  
        for (int i = 0; i < n; i++) {  
            for (int j = 1; j <= n; j++) {  
                System.out.print(j);  
            }  
            System.out.println();  
        }  
    }  
}
```

OUTPUT:



The screenshot shows the Eclipse IDE's Console window. The title bar includes 'Problems', 'Console', 'Terminal', and 'Coverage'. The console text shows the program's execution: it starts with '<terminated> J1406 [Java Application] /Applications/Eclipse.app/Contents/Eclipse', followed by the printed string 'Incrementing', and then five lines of the number '12345' printed on separate lines.

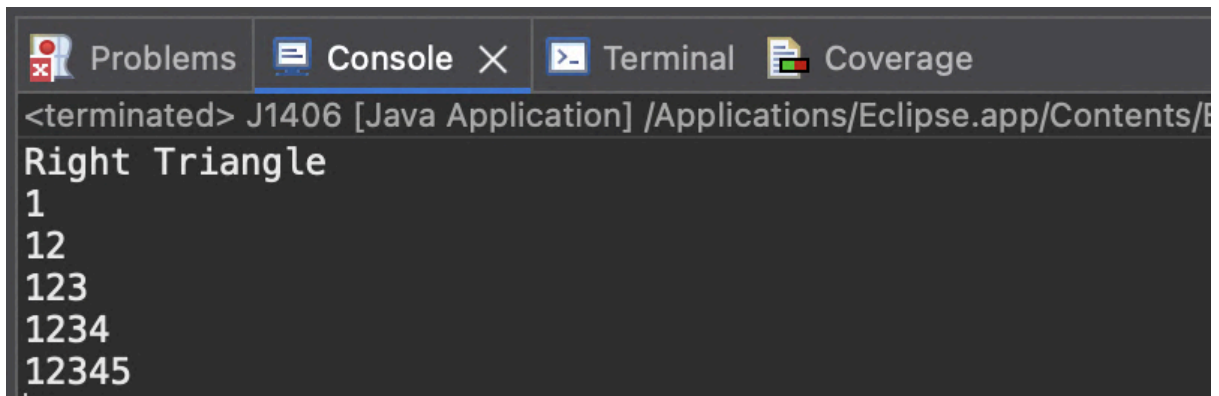
```
<terminated> J1406 [Java Application] /Applications/Eclipse.app/Contents/Eclipse  
Incrementing  
12345  
12345  
12345  
12345  
12345
```


- **RIGHT TRIANGLE:**

CODE:

```
public class J1406 {  
    public static void main(String[] args) {  
        System.out.println("Right Triangle");  
        int n = 5;  
        for (int i = 1; i <= n; i++) {  
            for (int j = 1; j <= i; j++) {  
                System.out.print(j);  
            }  
            System.out.println();  
        }  
    }  
}
```

OUTPUT:



The screenshot shows the Eclipse IDE interface with the 'Console' tab selected. The output of the Java application is displayed, showing the text 'Right Triangle' followed by five lines of numbers forming a right-angled triangle: '1', '12', '123', '1234', and '12345'. The console title bar indicates the application is 'J1406 [Java Application]' and the path is '/Applications/Eclipse.app/Contents/E'.

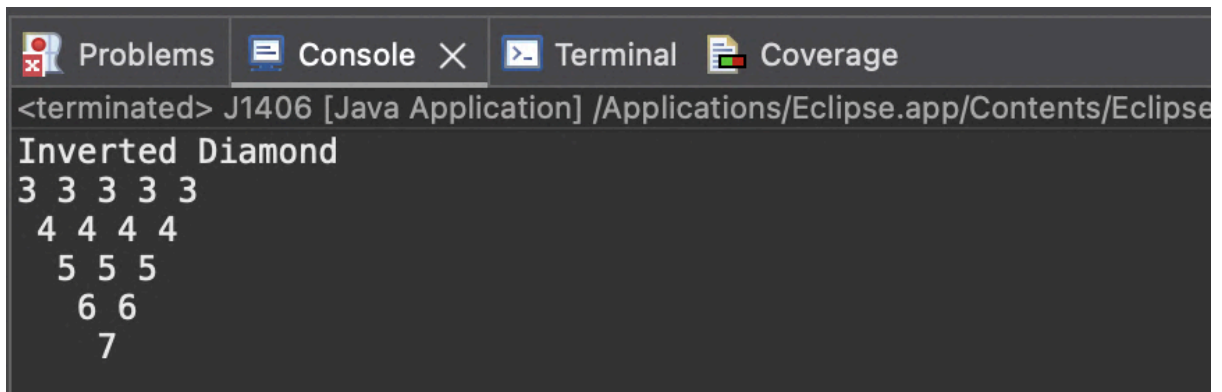
```
<terminated> J1406 [Java Application] /Applications/Eclipse.app/Contents/E  
Right Triangle  
1  
12  
123  
1234  
12345
```


- INVERTED DIAMOND:

CODE:

```
public class J1406 {  
    public static void main(String[] args) {  
        System.out.println("Inverted Diamond");  
        for (int i = 3; i <= 7; i++) {  
            for (int j = 0; j < i - 3; j++) {  
                System.out.print(" ");  
            }  
            for (int j = 0; j < 8 - i; j++) {  
                System.out.print(i + " ");  
            }  
            System.out.println();  
        }  
    }  
}
```

OUTPUT:



The screenshot shows the Eclipse IDE interface with the Console tab selected. The console output displays the text "Inverted Diamond" followed by five lines of numbers forming an inverted diamond shape. The first line has five '3's, the second has four '4's, the third has three '5's, the fourth has two '6's, and the fifth has one '7'.

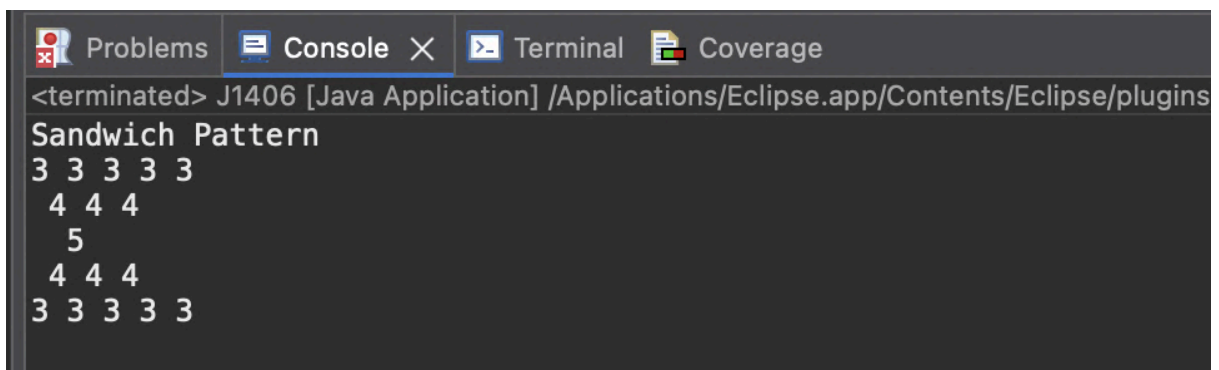
```
<terminated> J1406 [Java Application] /Applications/Eclipse.app/Contents/Eclipse  
Inverted Diamond  
3 3 3 3 3  
4 4 4 4  
5 5 5  
6 6  
7
```

- SANDWICH PATTERN:

CODE:

```
public class J1406 {  
    public static void main(String[] args) {  
        System.out.println("Sandwich Pattern");  
        for (int i = 3; i <= 5; i++) {  
            for (int j = 0; j < i - 3; j++) {  
                System.out.print(" ");  
            }  
            for (int j = 0; j < 2 * (5 - i) + 1; j++) {  
                System.out.print(i + " ");  
            }  
            System.out.println();  
        }  
        for (int i = 4; i >= 3; i--) {  
            for (int j = 0; j < i - 3; j++) {  
                System.out.print(" ");  
            }  
            for (int j = 0; j < 2 * (5 - i) + 1; j++) {  
                System.out.print(i + " ");  
            }  
            System.out.println();  
        }  
    }  
}
```

OUTPUT:



The screenshot shows the Eclipse IDE interface with the 'Console' tab selected. The output of the Java application 'J1406' is displayed, showing the title 'Sandwich Pattern' followed by a pattern of numbers. The pattern consists of two identical blocks of five lines each. The first block starts with '3 3 3 3 3' and ends with '3 3 3 3 3'. The second block starts with '4 4 4' and ends with '3 3 3 3 3'. The numbers are centered and separated by spaces.

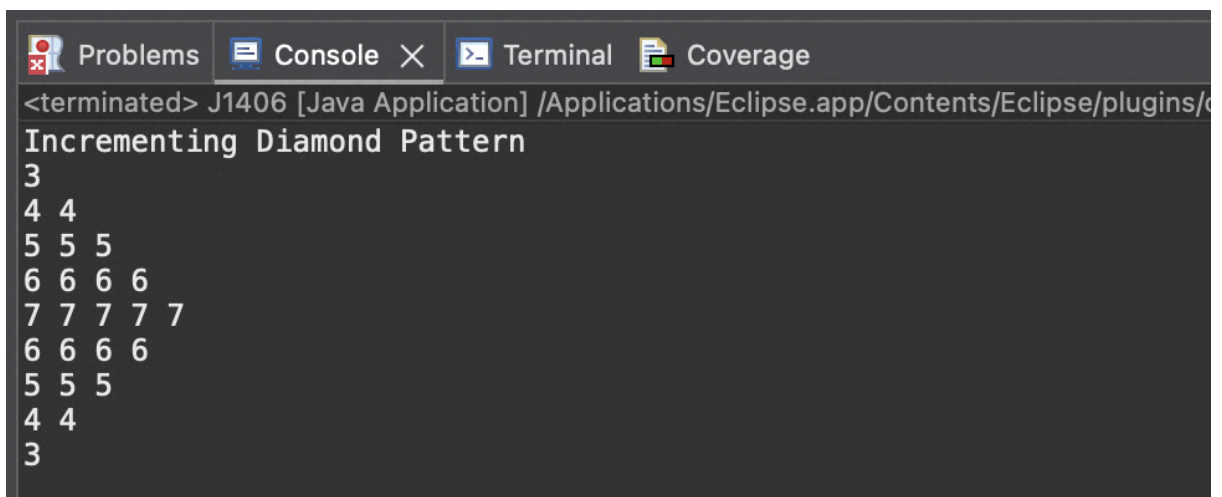
```
<terminated> J1406 [Java Application] /Applications/Eclipse.app/Contents/Eclipse/plugins  
Sandwich Pattern  
3 3 3 3 3  
4 4 4  
5  
4 4 4  
3 3 3 3 3  
4 4 4  
5  
4 4 4  
3 3 3 3 3
```


- INCREMENTING DIAMOND PATTERN:

CODE:

```
public class J1406 {  
    public static void main(String[] args) {  
        System.out.println("Incrementing Diamond Pattern");  
        for (int i = 3; i <= 7; i++) {  
            for (int j = 0; j < i - 2; j++) {  
                System.out.print(i + " ");  
            }  
            System.out.println();  
        }  
        for (int i = 6; i >= 3; i--) {  
            for (int j = 0; j < i - 2; j++) {  
                System.out.print(i + " ");  
            }  
            System.out.println();  
        }  
    }  
}
```

OUTPUT:



The screenshot shows the Eclipse IDE interface with the Console tab selected. The output of the Java program is displayed in the console window. The title bar of the console shows "Problems", "Console", "Terminal", and "Coverage". The output text is as follows:

```
<terminated> J1406 [Java Application] /Applications/Eclipse.app/Contents/Eclipse/plugins/c  
Incrementing Diamond Pattern  
3  
4 4  
5 5 5  
6 6 6 6  
7 7 7 7 7  
6 6 6 6  
5 5 5  
4 4  
3
```